

Path-finding Hardware Accelerator

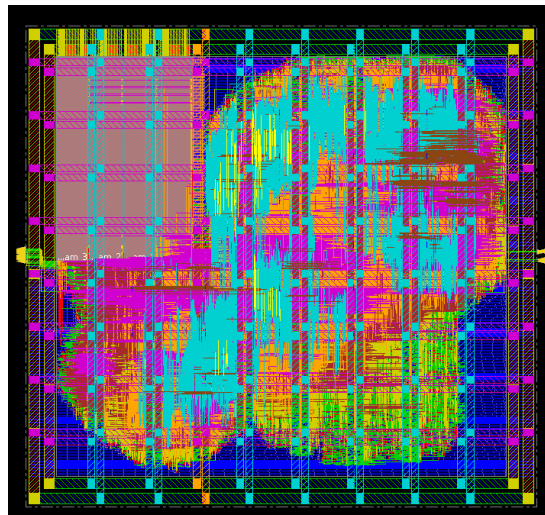
DIGITAL VLSI SYSTEMS ON CHIP

ET4351

Authors:

Navya Agarwal (6187188)
Siddharth Patil (6138667)
Priya Chittimuri (6168558)
Vagif Abdulla (Employee: 935242)

May 5, 2026



Contents

1	Introduction	2
2	Architecture and Design Methodology	2
2.1	Algorithm Overview	2
2.2	Design Specifications	3
2.2.1	Architecture Overview	3
2.2.2	Functional Specifications	4
2.3	Bottlenecks and Optimizations	4
2.3.1	Single pair path computation	4
2.3.2	Parallelizing the update of distance array	5
2.3.3	Dual Dijkstra instances (not implemented in the final design)	5
2.3.4	Parallelizing the global minimum finder (not implemented in the final design)	5
2.3.5	Memory optimization (not implemented in the final design)	5
3	Implementation	6
3.1	Optimization Strategy	6
3.1.1	Design Stage	6
3.1.2	Floorplan Stage	6
3.1.3	Placement Stage	7
3.2	Sign-off Verification	7
3.3	Summary	7
4	Performance Metrics and Validation	7
5	Post-layout simulation	8
5.0.1	Accelerator reset through firmware	9
5.0.2	Accelerator communicates properly with the firmware	9
5.0.3	Accelerator sends all paths consecutively and then sets the done flag which is held till the accelerator is disabled by the firmware	9
5.0.4	Valid signal high for each path and low after a single path	9
5.0.5	Start node from firmware incrementing	9
6	Conclusion	9
	Appendix A	9
	Appendix B	10

1 Introduction

This work presents the design and implementation of a hardware accelerator for efficient shortest path computation in graphs. The SOC uses a PicoRV32 RISC-V core along with a custom accelerator and 1kB SRAM. The accelerator is developed to interface with the PicoRV32 RISC-V core via UART. We compared multiple standard pathfinding algorithms and concluded that for the current requirements of the project Dijkstra’s algorithm is the best choice. Since the baseline provided to us was an implementation of the Dijkstra’s algorithm, we chose to build upon and improve it, as opposed to starting from scratch. This decision was partly motivated by the multiple areas of possible improvements we were able to identify in the implementation quite early on in the group project, offering us an opportunity to be more creative and innovative – much akin to real-world industry environments where engineers are often tasked to extend and optimize existing solutions. This approach not only enhanced our understanding of Dijkstra’s algorithm but also enabled us to discover its bottlenecks and explore more advanced optimization methods such as parallelization to alleviate them, which we were successful in doing so. Furthermore, we had more time to experiment with the place-and-route and simulation scripts, aiming to reduce power consumption and enhance energy efficiency, thus making our final implementation more suitable for practical hardware deployment. The accelerator design was subject to a set of requirements as listed:

Requirement	Specification	Final Results
Area	$513 \times 513 \mu\text{m}^2$	$513 \times 478.8 \mu\text{m}^2$
Total Latency	$< 47.76 \text{ ms at } 12 \text{ MHz}$	21.71 ms
Clock Cycles	$< 573 \times 10^3 \text{ cycles}$	260×10^3
Power	0.407 mW	0.408 mW
Energy	$< 19.37 \mu\text{J}$	$1.36 \mu\text{J}$
Timing	Clean setup and hold timing reports	✓
DRC/LVS	Clean connectivity, geometry, and antenna reports	✓

Table 1: Design Requirements Summary and Key Results

2 Architecture and Design Methodology

2.1 Algorithm Overview

In order to compute the shortest path from any node to any other node in the graph as quickly as possible several algorithms like Floyd Warshall, A* algorithm, Johnson’s algorithm and Bellman Ford were considered. While these algorithms were some of the best to compute the shortest path, it was essential to consider the specific characteristics of the given graph and evaluate their suitability in the context of hardware implementation. Based on these considerations, the algorithms were evaluated as follows:

- The A* search algorithm is known for its efficiency and accuracy. However, this approach introduces additional complexity and demands more computational resources due to the heuristic evaluation at each step.
- The Bellman–Ford algorithm supports graphs with negative edge weights and can detect negative cycles, but these capabilities are not necessary in the present case. Additionally, it was found to be inefficient on sparse graphs due to its higher time complexity.
- The Floyd–Warshall algorithm offers a robust all-pairs shortest path solution, particularly useful when negative edge weights are present—though this is not required in the present case. Similar to Bellman–Ford, its cubic time complexity makes it less suitable for large and sparse graphs.
- Johnson’s algorithm, while efficient for sparse graphs with negative weights, introduces unnecessary complexity in this context due to its initial reweighting step using Bellman–Ford. Since the given graph is sparse and does not contain negative weights, the overhead of this preprocessing phase outweighs its benefits.

Given these considerations, Dijkstra's algorithm stands out as the most appropriate choice, as it is well-suited for sparse graphs with non-negative edge weights.

At the software level, Dijkstra's algorithm is efficiently implemented using a min-heap priority queue and an adjacency list to store neighbors and edge weights. The algorithm initializes all distances as infinity, except the source (set to zero), and iteratively selects the node with the smallest tentative distance, relaxing its neighbors. This continues until all nodes are processed or the destination is reached. With a min-heap, the time complexity is $O((V + E) \log V)$ where V is the number of vertices and E is the number of edges.

2.2 Design Specifications

2.2.1 Architecture Overview

The SoC architecture, the memory map, and the accelerator architecture were kept the same. Majority of changes were implemented in the finite state machine (Fig. 2) of the algorithm and the firmware.

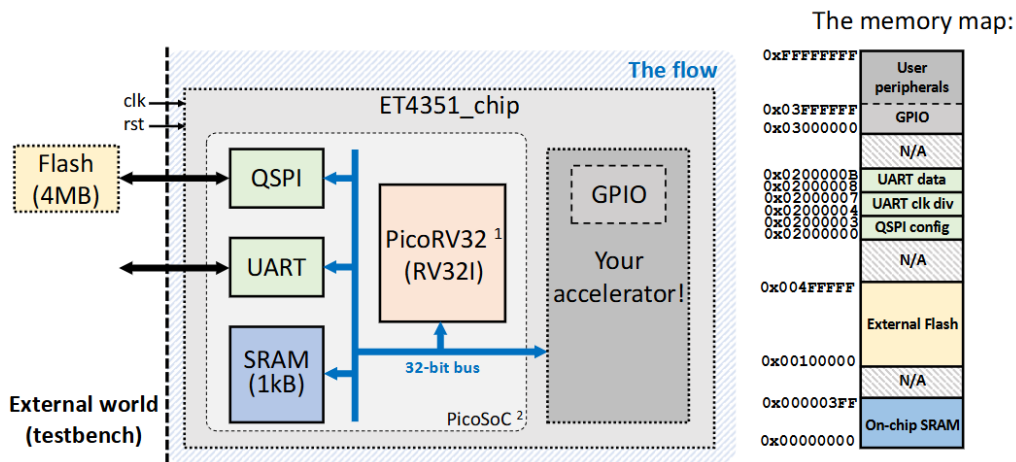


Figure 1: System-on-chip Architecture

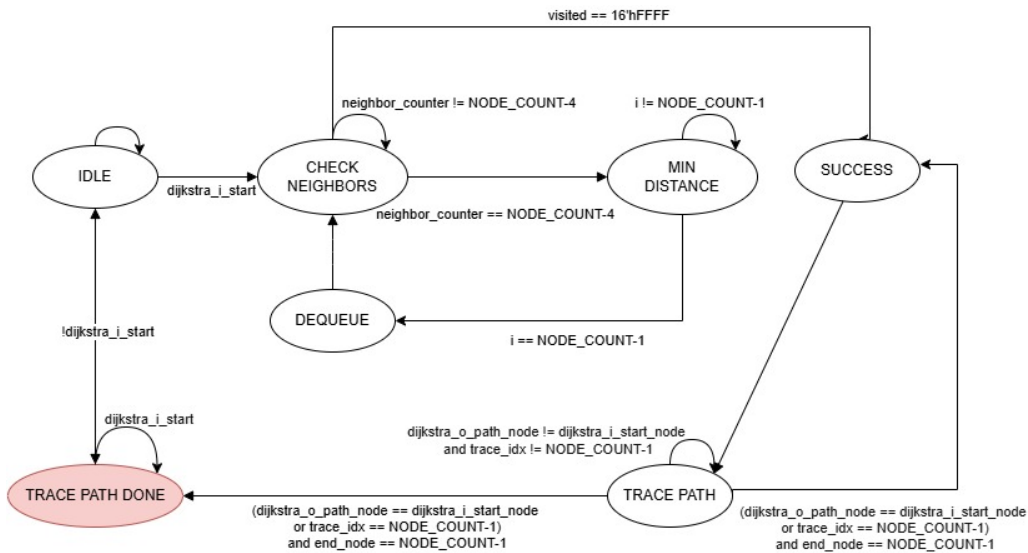


Figure 2: FSM

- **IDLE**: Initial stage which initializes all the data structures.
- **CHECK NEIGHBORS**: Iterates over all adjacent nodes four at a time and updates the respective distances. The state transitions when all neighbors have been checked.
- **MIN DISTANCE**: Identifies the unvisited node with the minimum tentative distance. The process continues until all nodes are considered.

- **DEQUEUE:** Mark current node as visited and prepare to process its neighbors.
- **SUCCESS:** Reached when all nodes are visited, indicating that all the shortest paths have been computed.
- **TRACE PATH:** Reconstructs the shortest paths from the destination back to the source using previously recorded parent nodes.
- **TRACE PATH DONE:** The final state is entered when the trace is complete, i.e., all the end nodes are covered.

2.2.2 Functional Specifications

1. Accelerator is reset from the firmware.
2. Accelerator receives the start node and then the enable signal from the firmware.
3. Accelerator sends to the testbench all shortest paths for a start node consecutively.
4. Accelerator sends `accel_o_path_node_valid` high to the testbench when sending each path.
5. Accelerator sends `accel_o_path_node_valid` low after each path.
6. After sending all shortest paths for each `start_node` accelerator sets the done flag.
7. The done flag remains high till the accelerator receives the disable signal from the firmware.
8. The firmware receives the done flag from the accelerator and then sends the disable signal to the accelerator.
9. Accelerator receives start node after receiving the disable signal.
10. Accelerator receives all the start nodes consecutively.

2.3 Bottlenecks and Optimizations

The baseline implementation of Dijkstra's algorithm was unoptimized and therefore several optimization techniques were explored to reduce latency and improve its suitability for hardware acceleration.

2.3.1 Single pair path computation

Dijkstra's algorithm is optimized to find the shortest paths from a single source to all reachable nodes in one pass. Running it once is generally more efficient than restarting it for each target, as the algorithm already explores nodes in order of increasing distance. Stopping early offers little computational benefit unless the target is very close to the source, since many nodes must still be processed to guarantee optimality.

To increase the efficiency of the computation the following changes were made

- Removed the *for* loop in firmware for specifying the end node
- Changed the state change conditions in the FSM to generate output only after distances for all the nodes updated. The condition was initially to generate output when the specified end node is reached.
- Changed output generation to send multiple paths consecutively instead of a single path per execution.

These changes resulted in significant reduction in the execution clock cycles. Initially it took 349291 clock cycles for the accelerator to finish the entire operation for all the nodes. After the changes it came down to 45042 clock cycles giving an 87% improvement. There are several reasons for the latency improvement including:

- Overhead of executing a loop in software removed.
- Redundant calculations for each path eliminated.
- Overhead of firmware enabling and disabling accelerator between operations reduced (UART communications).

2.3.2 Parallelizing the update of distance array

While the core of Dijkstra's algorithm is inherently sequential, performance improvements can be achieved by parallelizing parts of the distance update step. When the algorithm updates distances to the neighbors of the current node, each neighbor's tentative distance can be updated in parallel. Potentially all the node distances can be updated in parallel but the bottleneck comes in fetching the data from the accelerator memory. As we were already fetching data for four neighbors at a time parallelization was implemented such that distances for four neighbors would be updated in one clock cycle. This reduces the clock cycles from 16 to 4 for the CHECK_NEIGHBORS state. Since this state is visited around 15 times per node and we run the entire algorithm for 15 nodes it should give the reduction of at least $(16 - 4) \times 15 \times 15 = 2700$ clock cycles. To update the distance matrix in parallel the following changes were made

- Stored the node weights and temporary distances for 4 neighbors at a time.
- Updated distances matrix for 4 neighbors at a time in the CHECK_NEIGHBORS state.

These changes further reduced the clock cycles from 45042 to 40167 which is more than theoretically calculated. This is another 10% reduction in the latency.

2.3.3 Dual Dijkstra instances (not implemented in the final design)

One of the first optimizations considered was to implement two instances of Dijkstra such that each one performed the algorithm for alternating nodes and this algorithm is further explained in the Appendix 6. This would reduce the latency significantly as 2 nodes can be processed concurrently. However, implementing such an optimization would require substantial area. In this case the tradeoff between area and speed was not favorable and did not justify the marginal gain in throughput for the given constraints.

2.3.4 Parallelizing the global minimum finder (not implemented in the final design)

The MIN_DISTANCE state in the Dijkstra implementation checks for the next minimum distance amongst all the unvisited nodes. This process involves a lot of comparisons and attempts were made to parallelize this comparison using for loops. However, this resulted in a timeout error, meaning that the circuit generated for the comparisons introduced too much delay to be handled in a single clock cycle. As a result, the baseline sequential comparison method was retained to ensure design feasibility. If successful, this optimization would have saved the same number of clock cycles as in 2.3.2 (due to the same number of state transitions), but would have been significantly more challenging to implement due to the higher complexity without using pipelining or designing a faster comparator. Therefore, we decided against spending effort on parallelizing the global minimum finder in our final design.

2.3.5 Memory optimization (not implemented in the final design)

Once the sequential update of distance array was done, efforts were made to increase the number of neighbors from four to higher. Expanding this to accommodate all possible neighbors (i.e., the full number of nodes) could significantly reduce the number of clock cycles required for distance updates, thereby improving performance. However, this change would require architectural modifications (which were advised against in the code 17) to support higher memory access concurrency and therefore was not implemented.

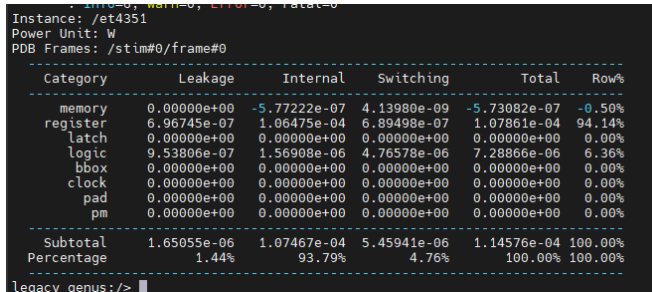
3 Implementation

3.1 Optimization Strategy

The implementation process was carefully customized and optimized using scripting at every key phase of the digital design workflow. Every adjustment was driven by precise design goals—specifically, lowering area and power consumption, meeting timing requirements, and maintaining a density of about 70%. The following section outlines the specific modifications applied in each stage:

3.1.1 Design Stage

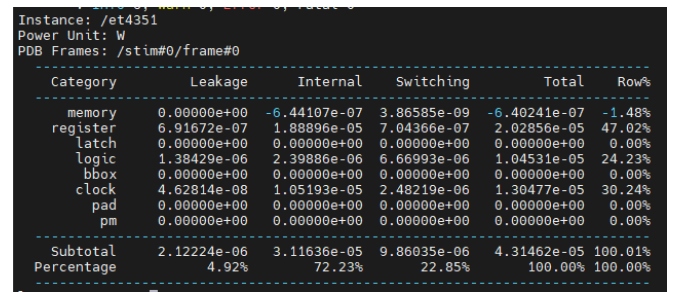
Clock gating reduces power consumption by reducing unnecessary transistor switching. After design synthesis stage, the final design with clock gating enabled reported a power consumption of 0.0431 mW, and the design without clock gating reported a power consumption of 0.114 mW. Based on these numbers, the implementation scripts were optimized for clock gating by enabling high efforts in the placement stage, which reported a final power consumption of 0.503 mW. And the implemented design without clock gating reported a power of 0.401 mW. Hence, based on the new power results, the design without clock gating enabled has been selected as the final design.



Instance: /et4351
Power Unit: W
PDB Frames: /stim#0/frame#0

Category	Leakage	Internal	Switching	Total	Row%
memory	0.00000e+00	-5.77222e-07	4.13980e-09	-5.73082e-07	-0.50%
register	6.96745e-07	1.06475e-04	6.89498e-07	1.07861e-04	94.14%
latch	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
logic	9.53806e-07	1.56908e-06	4.76578e-06	7.28866e-06	6.36%
bbox	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
clock	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
pad	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
pm	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
Subtotal	1.65055e-06	1.07467e-04	5.45941e-06	1.14576e-04	100.00%
Percentage	1.44%	93.79%	4.76%	100.00%	100.00%

(a) Design without Clock Gating



Instance: /et4351
Power Unit: W
PDB Frames: /stim#0/frame#0

Category	Leakage	Internal	Switching	Total	Row%
memory	0.00000e+00	-6.44107e-07	3.86585e-09	-6.40241e-07	-1.48%
register	6.91672e-07	1.88896e-05	7.04366e-07	2.02856e-05	47.02%
latch	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
logic	1.38429e-06	2.39886e-06	6.66933e-06	1.04531e-05	24.23%
bbox	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
clock	4.62814e-08	1.05193e-05	2.48219e-06	1.30477e-05	30.24%
pad	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
pm	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
Subtotal	2.12224e-06	3.11636e-05	9.86035e-06	4.31462e-05	100.01%
Percentage	4.92%	72.23%	22.85%	100.00%	100.00%

(b) Design with Clock Gating

Figure 3: Power Reports after Synthesis

3.1.2 Floorplan Stage

Customized the core height and width to meet the grid placement guidelines and to decrease the overall core area and thus reducing cost of production. The base design had a core dimension of $513 \times 513 \mu\text{m}^2$ with a density of 66%. Based on this and the characteristics of the filler cell FEED1, two alternative core size configurations were evaluated. The numbers below have been selected as they are an integer multiple of the FEED1 filler cell dimensions of 0.2 BY 1.71:

- $513 \times 478.8 \mu\text{m}^2$, which resulted in a density of 71%
- $478.8 \times 478.8 \mu\text{m}^2$, which resulted in a density of 78.8%

The design with the smallest core $478.8 \times 478.8 \mu\text{m}^2$ exhibited 8 glitch violations during timing analysis, making it unsuitable for sign-off. As a result, the configuration with a core size of $513 \times 478.8 \mu\text{m}^2$ was selected for the final implementation, as it provided a balanced trade-off between area savings and acceptable density levels, without introducing critical timing issues.

An experiment was done by doubling the number of power lines and reducing the power line thickness to half. This experiment resulted in a negligible change in the final utilized area and a lower power consumption of 0.399 mW. We hypothesize that this change was due to the lowered length (and hence resistances) of wiring. Due to such a negligible change in results, these changes were not implemented in the final design.

3.1.3 Placement Stage

The design requirement calls for a low latency of the design and high power efficiency. To meet these requirements, Fig 4 shows the settings that were added to the setPlaceMode command in Innovus for better design optimization.

```
-place_global_timing_effort high
-place_global_activity_power_driven_effort high
-place_global_align_macro true
-place_global_clock_power_driven true
-place_global_clock_power_driven_effort high
```

Figure 4: Innovus setPlaceMode Settings

The given set of commands configures the global placement stage of the physical design flow to prioritize power efficiency, timing optimization, and macro alignment. By setting high effort levels for both power-driven and timing-driven placement, the tool is instructed to place cells in a way that minimizes dynamic power consumption and improves timing closure. The inclusion of these modes ensure that clock-related power is also considered, potentially reducing clock tree power by optimizing placement for lower capacitance. They enable the tool to align nearby macros of the same size, which helps create a more regular and structured floorplan, easing routing and reducing congestion.

3.2 Sign-off Verification

The finalized design did not show setup and hold violations after implementation. As shown in Fig 14, the design has no DRC and DRV violations apart from max_tran violations that are outside the scope of this course. The post-layout simulation also passes.

3.3 Summary

This section outlines the optimization strategy applied during the digital design flow, focusing on reducing power consumption, improving timing, and minimizing routing congestion. At the design stage, clock gating was evaluated but ultimately not used due to better overall results without it. In the floorplan stage, different core sizes were tested to balance area savings and routing congestion, with a $513 \times 478.8 \mu\text{m}^2$ configuration chosen for its optimal trade-off. During placement, various power and timing-driven settings were applied using Innovus commands to enhance placement quality. These settings prioritized timing closure, reduced dynamic and clock power, and improved macro alignment for a more structured and efficient layout.

4 Performance Metrics and Validation

	Baseline	2.3.1	2.3.2 with clock gating	2.3.2 without clock gating	3.1.2
Simulation clock cycles (accel)	349291	45042	40167	40167	40167
Total latency (ms)	47.76	22.11	21.71	21.71	21.71
Accel. latency (ms)	29.10	3.75	3.34	3.34	3.34
Accel. area (μm^2)	50322.564	51049.998	46211.382	50738.094	50188.158
Chip area usage (μm^2)	182911.381	182898.385	203773.723	181570.399	180457.531
Total Area (μm^2)	263169	263169	263169	263169	245624.4
Utilization	69.5%	69.5%	77%	66%	71.057%
Power (mW)	0.406592	0.4075	0.50337	0.40498409	0.408
Energy (μJ)	11.83	1.53	1.68	1.35	1.36
PPA (nJm^2)	595.41	78.01	77.69	68.63	68.39

Table 2: Comparison of design optimization metrics

Note: The column headings refer to the optimization sections discussed in the report.
The annotation coverage report is mentioned in Fig 14. In the table 2, the power number mentioned is for typical corner, the power numbers for slow and fast corners are given in Fig 15 and Fig 16 respectively.

5 Post-layout simulation

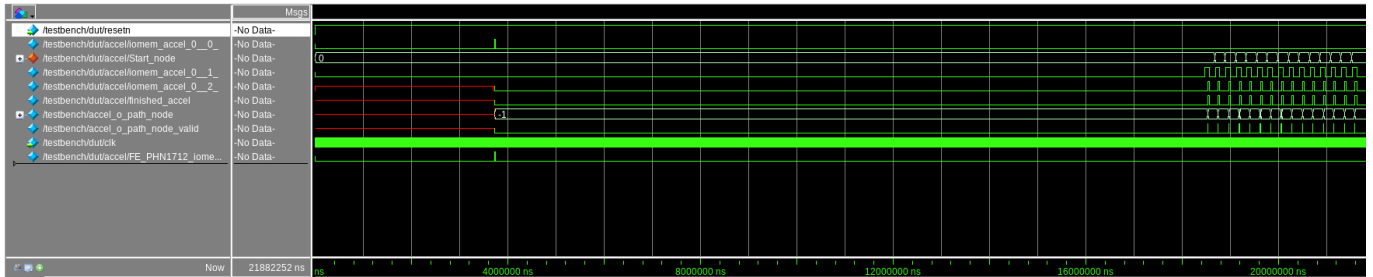


Figure 5: Accelerator Reset

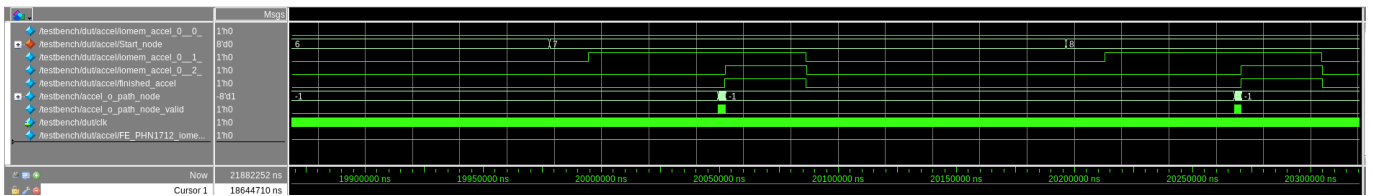


Figure 6: Firmware accelerator communication

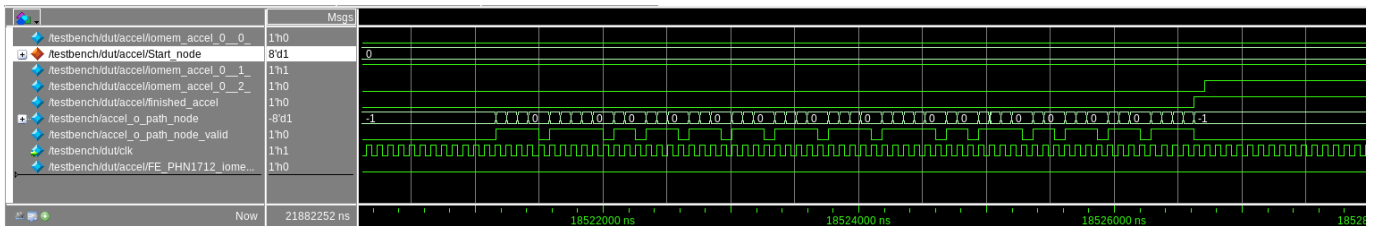


Figure 7: Waveform for start_node 0

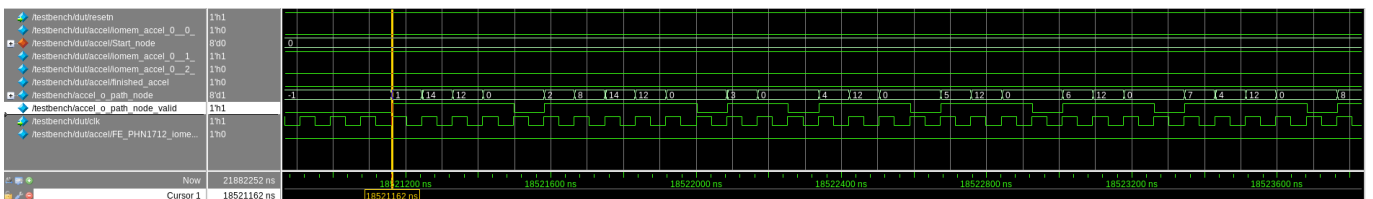


Figure 8: Consecutive paths

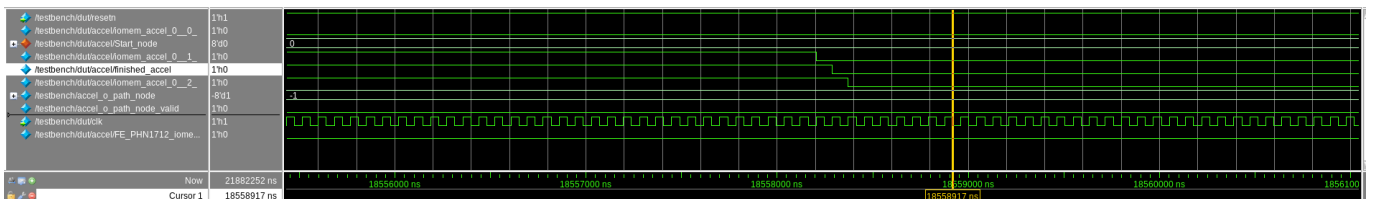


Figure 9: Done flag held till accelerator disabled

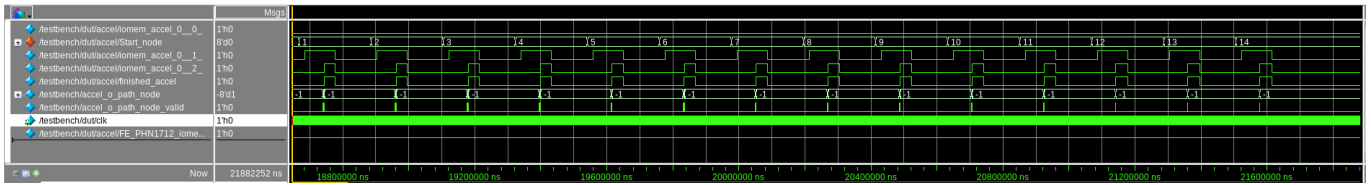


Figure 10: Consecutive start nodes

5.0.1 Accelerator reset through firmware

Covering functional specification (1) we can see in Fig. 5 the accelerator signals only being active after receiving the CSR bit for reset from firmware.

5.0.2 Accelerator communicates properly with the firmware

Covering functional specification (2),(8) and (9) we can see in the Fig. 6 after the done flag (iomem_accel_0_2_) is set the accelerator is disabled by setting the enable flag (iomem_accel_0_1_) 0. The firmware then sends the next start_node 7 and then the enable flag (iomem_accel_0_1_) as 1.

5.0.3 Accelerator sends all paths consecutively and then sets the done flag which is held till the accelerator is disabled by the firmware

Covering functional specifications (3),(6) and (7) we can see in Fig. 7 for star_node 0 there are 15 output paths sent and then the done flag (iomem_accel_0_2_) is set. From Fig. 8 we can see these paths are consecutive. From Fig. 9 we can see that the done flag is held till the firmware sets the enable flag (iomem_accel_0_1_) 0. This is to ensure proper handshaking as the firmware disables the accelerator only after receiving the done flag.

5.0.4 Valid signal high for each path and low after a single path

Covering functional specifications (4),(5) we can see in Fig. 8 the accel_o_path_node_valid signal stays high when the shortest path is being sent and then goes low.

5.0.5 Start node from firmware incrementing

Covering functional specification (10) we can see in Fig. 10 the start nodes being sent consecutively from the firmware.

6 Conclusion

This project successfully demonstrates the design and optimization of a hardware accelerator for efficient shortest path computation using Dijkstra's algorithm. Through careful architectural decisions and systematic optimizations, significant improvements were achieved in both latency and energy efficiency, thereby meeting the specified design requirements. Specifically for the accelerator, key performance metrics include a reduction in clock cycles by nearly 88.5%, a drop in energy consumption by 88.5%, and a reduction in area by 6.67%, clearly highlighting the effectiveness of the implemented enhancements

Appendix A

Dual Dijkstra Instance

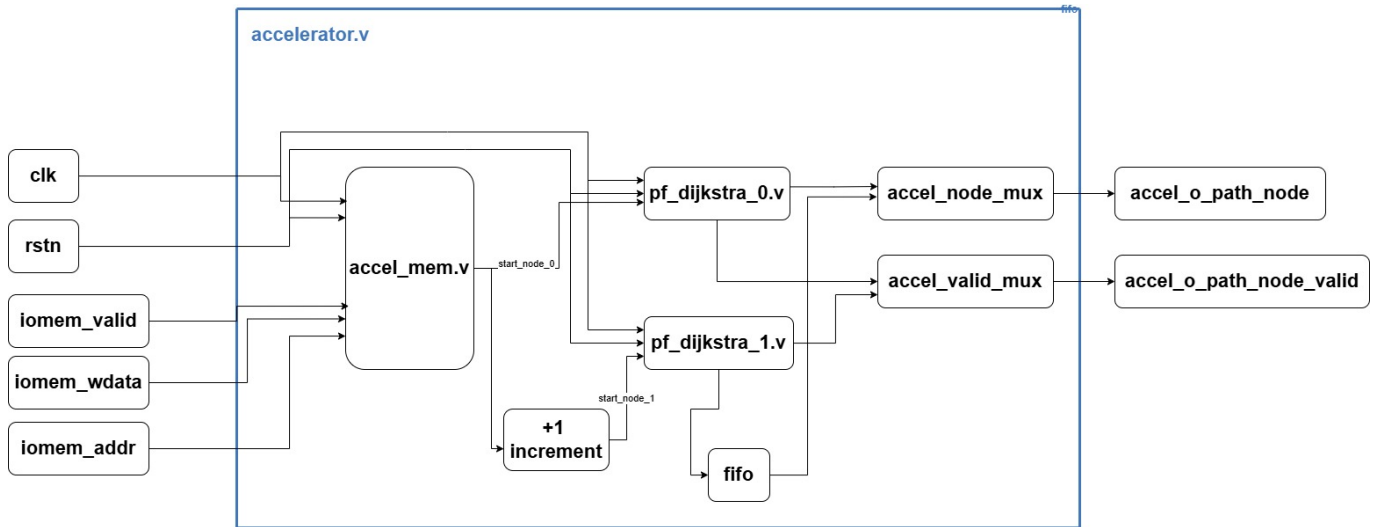


Figure 11: Dual Dijkstra Architecture

The presented architecture in Fig 11 outlines a hardware accelerator concept aimed at executing Dijkstra's shortest path algorithm in parallel from two adjacent starting nodes to improve throughput and efficiency. The design features two identical Dijkstra processing units 'pf_dijkstra_0' and 'pf_dijkstra_1' that share access to a centralized memory module 'accel_mem.v'. This memory module is responsible for storing the graph data and providing the initial start node for the computation. The first Dijkstra instance receives 'start_node_0' directly, while the second instance operates on 'start_node_1', which is simply an incremented version of the first node. On paper, this dual-instance approach was evaluated and showed strong potential for speeding up graph traversal by enabling parallel computation of shortest paths from neighboring nodes. However, the actual implementation of this architecture was not completed due to time constraints. One of the primary challenges encountered was designing and testing the output multiplexing logic. The two Dijkstra cores output node values and valid flags independently, requiring carefully controlled multiplexers 'accel_node_mux' and 'accel_valid_mux' to combine their results into a single output stream. The complexity of ensuring proper timing and avoiding contention made this a difficult task. Additionally, the initial design included a FIFO buffer to handle the output of the second instance, but further consideration revealed that this might be unnecessary. Instead, the existing 'accel_mem.v' memory structure could potentially be reused to store the output data from the second Dijkstra instance, simplifying the architecture and reducing hardware overhead. Overall, while the concept appears promising and is backed by sound theoretical evaluation, completing the design would require additional development time, particularly for refining the output arbitration logic and optimizing memory reuse. These changes are expected to make the accelerator more resource-efficient and scalable for larger graphs or more parallel instances.

Appendix B

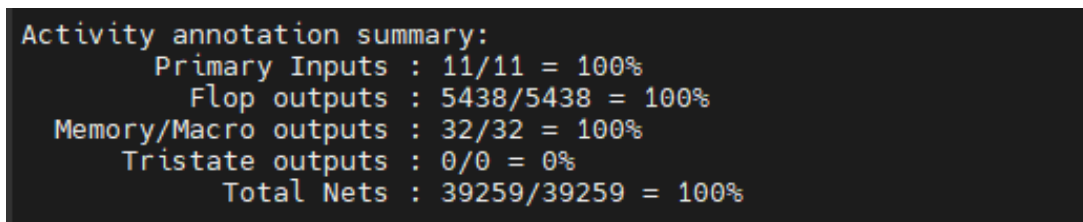


Figure 12: VCD Annotation Coverage

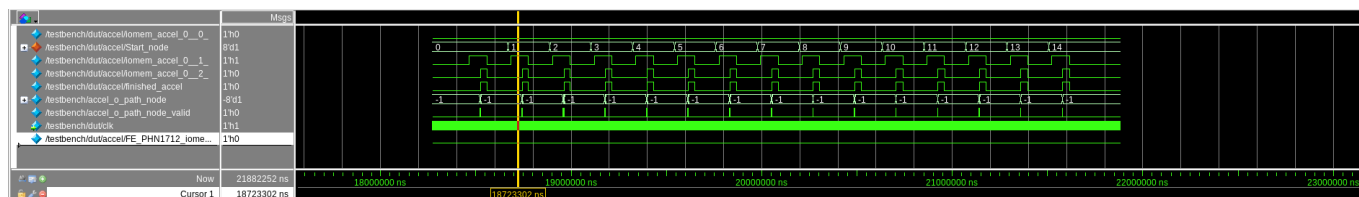


Figure 13: Activity annotation waveform

Setup mode	all	reg2reg	default
WNS (ns):	33.997	38.389	33.997
TNS (ns):	0.000	0.000	0.000
Violating Paths:	0	0	0
All Paths:	7760	7746	235

DRVs	Real	Total
	Nr nets (terms)	Nr nets (terms)
max_cap	0 (0)	0 (0)
max_tran	109 (302)	113 (310)
max_fanout	0 (0)	0 (0)
max_length	0 (0)	0 (0)

Density: 71.057%
 (100.000% with Fillers)
 Total number of glitch violations: 0

Figure 14: Post Route Summary

```

*      report_power -outfile powerReports/power_setup_slow_vdd1v0_basicCells.rpt
*

```

Total Power

Total Internal Power:	0.12248501	75.3179%
Total Switching Power:	0.03736819	22.9783%
Total Leakage Power:	0.00277089	1.7039%
Total Power:	0.16262408	

Group	Internal Power	Switching Power	Leakage Power	Total Power	Percentage (%)
Sequential	0.1033	0.0002419	0.0006598	0.1042	64.08
Macro	1.475e-06	5.128e-06	0	6.603e-06	0.00406
IO	0	0	3.511e-08	3.511e-08	2.159e-05
Combinational	0.004539	0.004835	0.002069	0.01144	7.036
Clock (Combinational)	0.01464	0.03229	4.183e-05	0.04697	28.88
Clock (Sequential)	0	0	0	0	0
Total	0.1225	0.03737	0.002771	0.1626	100

Rail	Voltage	Internal Power	Switching Power	Leakage Power	Total Power	Percentage (%)
VDD	0.9	0.1225	0.03737	0.002771	0.1626	100

Clock	Internal Power	Switching Power	Leakage Power	Total Power	Percentage (%)
clk	0.01464	0.03229	4.183e-05	0.04697	28.88
Total (excluding duplicates)	0.01464	0.03229	4.183e-05	0.04697	28.88

Figure 15: Power Setup: slow_vdd1v0_basicCells

```

*      report_power -outfile powerReports/power_hold_fast_vdd1v2_basicCells.rpt
*

```

Total Power

Total Internal Power:	0.29280884	74.7981%
Total Switching Power:	0.08226927	21.0157%
Total Leakage Power:	0.01638763	4.1862%
Total Power:	0.39146574	

Group	Internal Power	Switching Power	Leakage Power	Total Power	Percentage (%)
Sequential	0.2499	0.0005273	0.004007	0.2544	64.98
Macro	1.249e-05	1.112e-05	0	2.361e-05	0.006031
IO	0	0	3.618e-07	3.618e-07	9.242e-05
Combinational	0.007685	0.007938	0.01223	0.02785	7.115
Clock (Combinational)	0.03525	0.07379	0.0001504	0.1092	27.89
Clock (Sequential)	0	0	0	0	0
Total	0.2928	0.08227	0.01639	0.3915	100

Rail	Voltage	Internal Power	Switching Power	Leakage Power	Total Power	Percentage (%)
VDD	1.32	0.2928	0.08227	0.01639	0.3915	100

Clock	Internal Power	Switching Power	Leakage Power	Total Power	Percentage (%)
clk	0.03525	0.07379	0.0001504	0.1092	27.89
Total (excluding duplicates)	0.03525	0.07379	0.0001504	0.1092	27.89

Figure 16: Power Hold: fast_vdd1v2_basicCells

```

// IT IS NOT RECOMMENDED TO CHANGE THESE PARAMETERS
localparam MEM_WIDTH = 32;
localparam MEM_DEPTH = (NODE_COUNT*NODE_COUNT+3)/4;
localparam ADDR_WIDTH = $clog2(MEM_DEPTH);

```

Figure 17: Caution